

Universidad Autónoma Metropolitana

Unidad Azcapotzalco

División de Ciencias Básicas e Ingeniería
Licenciatura en Ingeniería en Computación

Ejercicio de RMI: sistema distribuido de tablero gráfico de noticias

Práctica 5
Sistemas Distribuidos

Alumno

José Julian Esteban Nuñez
2203026584
al2203026584@azc.uam.mx

Profesor: José Alejandro Reyes Ortiz

Trimestre 26-O

21 de julio de 2026

1. Proyecto

El proyecto fue desarrollado en **Java** utilizando **Maven** ya que esto facilitó más el proyecto y la gestión de dependencias.

Código fuente

El código fuente de la práctica se encuentra en el repositorio [stbn27/cliente-servidor-sockets_java/p5](https://github.com/stbn27/cliente-servidor-sockets_java/p5). Contiene instrucciones para compilar y ejecutar el proyecto, así como un archivo `README.md` con información adicional.

1.1. Servidor

El **servidor** no tiene interfaz gráfica, y su ejecución se realiza desde la línea de comandos. Como dependencias usas las siguientes librerías:

- **distribuidos:common** - Contiene clases y utilidades compartidas entre el cliente y el servidor.
- **com.h2database:h2** - Base de datos en memoria utilizada para almacenar la información de los usuarios y sus contraseñas.
- **at.favre.lib:bcrypt** - Librería para el hash de contraseñas
- **org.slf4j:slf4j-api** - Interfaz de logging utilizada por el servidor.

La **base de datos** utilizada es **H2**, que es una base de datos en memoria. Esto significa que los datos se almacenan temporalmente mientras el servidor está en ejecución y se pierden al cerrar el servidor. Esto es útil para pruebas y desarrollo, pero no es adecuado para un entorno de producción donde se requiere persistencia de datos. Las **sentencias SQL** para crear las tablas necesarias se ejecutan en automático al iniciar el servidor, por lo que no es necesario ejecutar ningún script adicional para configurar la base de datos.

Las credenciales de acceso y variables se configuran en el archivo `server.properties`, los valores por defecto son los siguientes:

Código 1: Archivo de configuración del servidor (`server.properties`)

```
rmi.host=localhost
rmi.port=1099
rmi.object.port=1100
rmi.service=TableroNoticias
database.path=./data/tablero-noticias
session.timeout.minutes=30
```

Nota: Aunque **H2** permite acceder a la consola web para administrar la base de datos, en este proyecto no se pueden crear usuarios directamente desde ahí, ya que las contraseñas se almacenan encriptadas, si se quisiera crear un usuario usando el apartado de administración deshabilitar la encriptación de contraseñas en los archivos `DatabaseInitializer.java` y en `AutenticacionService.java` en dichos archivos ya está documentada las líneas respectivas.

1.2. Cliente

El cliente tiene una interfaz gráfica desarrollada con **swing**. La interfaz permite a los usuarios colocar las credenciales de acceso al servidor, ver las noticias(sin iniciar sesión) y realizar acciones como iniciar sesión, cerrar sesión y publicar noticias (si se ha iniciado sesión). La interfaz es sencilla y fácil de usar, con botones y campos de texto para interactuar con el servidor.

Las dependencias del cliente son las siguientes:

- **distribuidos:common** - Contiene clases y utilidades compartidas entre el cliente y el servidor.
- **com.formdev:flatlaf** - Librería para la apariencia de la interfaz gráfica, proporcionando un estilo moderno y consistente.

Cuando se ejecuta el cliente, en automático intenta conectarse al servidor utilizando la dirección y el puerto especificados en el archivo **ConexionDialog**. Si el servidor no está disponible, se muestra una ventana para que el usuario ingrese la dirección y el puerto correctos. Una vez conectado, el cliente puede enviar solicitudes al servidor y recibir respuestas, permitiendo a los usuarios interactuar con la aplicación de manera efectiva.

2. Corridas de ejecución

2.1. Servidor ejecución

El archivo donde se inicia la ejecución del servidor es **ServidorApplication.java**. Este archivo contiene el método **main** que se encarga de inicializar el servidor, configurar la base de datos y poner a disposición los servicios necesarios para que los clientes puedan conectarse y realizar operaciones.

En la consola muestra los **logs** de inicio, esto se aprecian en la Figura 1. Los **logs** muestran información sobre la configuración del servidor, la inicialización de la base de datos y la disponibilidad de los servicios.

```
2026-07-21 13:46:17,918 INFO [main] server.database.DatabaseManager - Base H2 disponible
2026-07-21 13:46:18,194 INFO [main] s.database.DatabaseInitializer - Esquema H2 inicializado
2026-07-21 13:46:18,487 INFO [main] s.bootstrap.ServidorApplication - Registry RMI creado en puerto 1099
2026-07-21 13:46:18,491 INFO [main] s.bootstrap.ServidorApplication - Servicio TableroNoticias registrado; host=localhost, registry=1099, objeto=1100
```

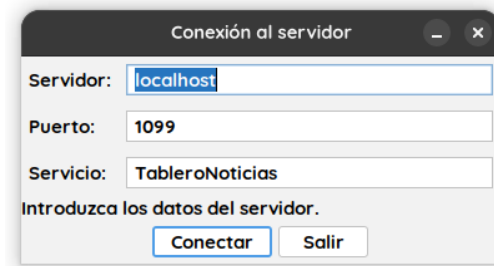
Figura 1: Logs de inicio del servidor

No se necesita configurar nada adicional para que el servidor funcione, ya que las sentencias SQL para crear las tablas necesarias se ejecutan automáticamente al iniciar el servidor. Esto significa que no es necesario ejecutar ningún script adicional para configurar la base de datos.

2.2. Cliente ejecución

El archivo donde se inicia la ejecución del cliente es **ClienteApplication.java**. Este archivo contiene el método **main** que se encarga de inicializar la interfaz gráfica, establecer la conexión con el servidor y permitir a los usuarios interactuar con la aplicación.

Si el servidor no esta activado y el cliente arranca en automático se muestra una ventana para que el usuario ingrese la dirección y el puerto correctos, como se muestra en la Figura 2. Una vez conectado, el cliente puede enviar solicitudes al servidor y recibir respuestas, permitiendo a los usuarios interactuar con la aplicación de manera efectiva.



Conexión al servidor

Servidor: localhost

Puerto: 1099

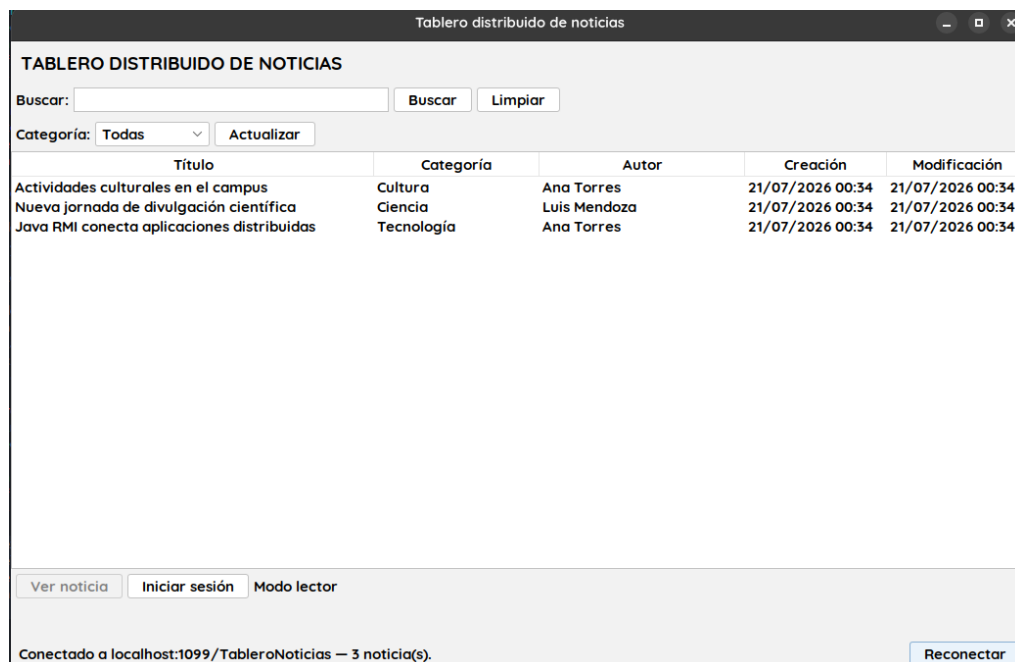
Servicio: TableroNoticias

Introduzca los datos del servidor.

Conectar Salir

Figura 2: Ventana de conexión del cliente

En caso de éxito o que las credenciales sean correctas, se muestra la ventana principal del cliente, como se muestra en la Figura 3. Esta ventana permite a los usuarios ver las noticias, iniciar sesión, cerrar sesión y publicar noticias (si se ha iniciado sesión).



Tablero distribuido de noticias

TABLERO DISTRIBUIDO DE NOTICIAS

Buscar: Buscar Limpiar

Categoría: Todas Actualizar

Título	Categoría	Autor	Creación	Modificación
Actividades culturales en el campus	Cultura	Ana Torres	21/07/2026 00:34	21/07/2026 00:34
Nueva jornada de divulgación científica	Ciencia	Luis Mendoza	21/07/2026 00:34	21/07/2026 00:34
Java RMI conecta aplicaciones distribuidas	Tecnología	Ana Torres	21/07/2026 00:34	21/07/2026 00:34

Ver noticia Iniciar sesión Modo lector

Conectado a localhost:1099/TableroNoticias — 3 noticia(s). Reconnectar

Figura 3: Ventana principal del cliente

Aquí el usuario sin necesidad de **iniciarsesión** puede ver las noticias publicadas, pero si desea publicar una noticia debe iniciar sesión con un usuario registrado. En caso de que el usuario no tenga una cuenta, debe solicitar al administrador del sistema que le cree una cuenta para poder iniciar sesión y publicar noticias.

Las funciones básicas de este apartado son las siguientes:

- **Ver noticias:** Permite a los usuarios ver las noticias publicadas sin necesidad de iniciar sesión.
- **Iniciar sesión:** Permite a los usuarios registrados iniciar sesión en la aplicación para poder

publicar noticias.

- **Cerrar sesión:** Permite a los usuarios buscar una noticia por su título usando el campo de búsqueda, tal y como se aprecia en la Figura 4, donde filtro por el término *Java*.
- **Publicar noticias:** Permite a los usuarios filtrar por categoría, fecha o autor.

Título	Categoría	Autor	Creación	Modificación
Java RMI conecta aplicaciones distribuidas	Tecnología	Ana Torres	21/07/2026 00:34	21/07/2026 00:34

Figura 4: Ventana de búsqueda de noticias en el tablero

Para que un usuario pueda publicar una noticia, debe iniciar sesión con un usuario registrado. Para ello solo presiona el botón *Iniciar sesión* y se muestra la ventana de inicio de sesión, como se aprecia en la Figura 5. En esta ventana el usuario debe ingresar su nombre de usuario y contraseña para poder iniciar sesión.

Usuario:

Contraseña:

Figura 5: Ventana de inicio de sesión del cliente

Si el usuario ingresa las credenciales incorrectas se muestra un mensaje de error, como se aprecia en la Figura ?? . En este caso, el usuario debe verificar sus credenciales y volver a intentar iniciar sesión.

3. Preguntas de reflexión

¿Qué ventajas ofrece RMI sobre sockets tradicionales?

RMI permite que el código cliente invoque métodos definidos en una interfaz Java aunque el objeto se encuentre en otro proceso o equipo. El programador trabaja con operaciones como 'listarNoticias()' o 'editarNoticia(...)', en vez de diseñar manualmente un formato de mensajes, delimitadores, códigos de operación y reglas para leer bytes parciales desde un socket.

El stub y el runtime abstraen el intercambio de mensajes. RMI serializa argumentos, resultados y excepciones; el Registry ofrece un mecanismo de localización; y 'RemoteException' hace visible que una llamada que parece ordinaria puede fallar por la red. Las interfaces remotas también constituyen un contrato verificable por el compilador: cliente y servidor conocen las firmas y los tipos compartidos.

Con sockets tradicionales existe mayor libertad sobre el protocolo, la representación binaria, el uso de memoria, las conexiones y la compatibilidad entre lenguajes. Esa libertad es valiosa para protocolos especializados o de alto rendimiento, pero obliga a implementar framing, serialización, despacho, correlación de respuestas, evolución del protocolo y manejo de errores. RMI reduce ese trabajo para un sistema Java a Java y hace la práctica más cercana al dominio.

La abstracción no elimina la naturaleza distribuida: una invocación puede tardar, fallar después de haber ejecutado parcialmente una acción o devolver una excepción. Además, RMI acopla ambos extremos a Java y a clases compatibles, su interoperabilidad es menor, la configuración de puertos/firewall requiere atención y la serialización debe tratarse como una frontera sensible. Por ello RMI es más productivo que sockets para este ejercicio homogéneo, pero no siempre es el transporte preferido para sistemas abiertos o políglotas.

¿Cómo se manejan múltiples peticiones simultáneas?

El runtime de RMI puede atender distintas invocaciones concurrentemente. La implementación no supone que los métodos se ejecuten uno tras otro ni sincroniza globalmente el objeto remoto. En su lugar, las clases de servicio y repositorio son esencialmente sin estado: los parámetros y variables de cada petición permanecen locales al hilo que la atiende.

Cada operación obtiene una conexión JDBC propia, y cada escritura delimita su transacción. De esa manera una solicitud no comparte un 'Connection', 'PreparedStatement' o 'ResultSet' mutable con otra. H2 coordina el acceso a sus datos y las transacciones evitan estados parciales. Las lecturas se mantienen independientes y no esperan un monitor global de Java.

Las sesiones sí son estado compartido, por lo que se almacenan en un 'ConcurrentHashMap'. La creación, consulta, expiración y eliminación de tokens se realizan con operaciones seguras de esa colección. Las noticias no se duplican en un mapa del servidor: H2 continúa siendo su única fuente de verdad.

Las pruebas de concurrencia previstas usan 'ExecutorService', 'CountDownLatch' y futuros para iniciar lectores, publicaciones y ediciones con suficiente simultaneidad. En particular, dos tareas que editan la misma versión deben producir exactamente un éxito y un conflicto, sin depender de pausas arbitrarias con 'Thread.sleep'.

¿Qué problemas enfrentaron al implementar consistencia de los datos?

El caso principal es la actualización perdida. Dos redactores pueden abrir la misma noticia, conservar la misma versión y enviar cambios distintos. Si el servidor ejecutara un 'UPDATE' solo

por identificador, el último reemplazaría al primero sin saber que trabajó sobre información obsoleta. El campo 'version' y una condición adicional en el 'WHERE' convierten esa carrera en un conflicto explícito.

También se deben coordinar lecturas mientras existen escrituras. Las transacciones hacen que una publicación o edición sea una unidad: no se expone una noticia con solo parte de sus campos modificados. El orden por fecha de modificación e identificador produce resultados repetibles incluso cuando dos filas comparten una marca temporal.

La consistencia no se limita a concurrencia. El servidor debe comprobar que el autor de la sesión sea propietario de la noticia, pues confiar en un botón deshabilitado permitiría modificar datos mediante otro cliente. Las claves foráneas impiden autores inexistentes; 'NOT NULL', usuario único y 'CHECK (version >= 1)' refuerzan invariantes; la validación limita textos y rechaza entradas vacías; y las consultas preparadas separan datos de SQL.

Finalmente, una excepción a mitad de una escritura debe provocar rollback. La dificultad está en traducir el fallo a una excepción de dominio comprensible sin ocultarlo en logs ni filtrar detalles internos. La solución separa el registro técnico del mensaje remoto y utiliza 'try-with-resources' para liberar recursos aun cuando la operación falla.

¿Cómo escalarías este sistema en un entorno real?

Primero separaría la lógica de negocio del transporte para conservarla al reemplazar RMI por gRPC o una API REST según los consumidores, requisitos de interoperabilidad y operación. Para clientes internos fuertemente tipados, gRPC permitiría contratos eficientes y multiplataforma; para integración web y ecosistemas amplios, HTTP/REST podría ser más accesible. La elección no sería automática: dependería de latencia, streaming, compatibilidad y herramientas disponibles.

H2 se sustituiría por una base administrada con pool de conexiones, migraciones, respaldos, réplicas y estrategia de recuperación. Varias instancias del servicio se colocarían detrás de un balanceador. Las sesiones dejarían de residir en un único proceso: podrían usarse tokens verificables de corta duración o un almacén compartido y altamente disponible. Las escrituras requerirían claves de idempotencia para soportar reintentos ambiguos.

Para cargas de lectura elevadas podrían incorporarse cachés con invalidación explícita y réplicas de lectura. La publicación de eventos mediante mensajería desacoplaría indexación, auditoría y notificaciones. Un motor de búsqueda atendería consultas de texto completo, y WebSocket, Server-Sent Events o streaming proporcionarían notificaciones en tiempo real.

La operación necesitaría observabilidad: logs estructurados y centralizados, métricas, trazas distribuidas, tableros y alertas. La seguridad incluiría TLS mutuo o una terminación TLS confiable, proveedor de identidad, rotación de secretos, mínimos privilegios, auditoría y límites de solicitudes. Réplicas en zonas distintas, comprobaciones de salud, despliegues automatizados y pruebas de recuperación aportarían alta disponibilidad.

Ninguna de estas mejoras se implementa en la práctica porque añadiría infraestructura que ocultaría el objetivo: demostrar Java RMI, serialización, persistencia local, concurrencia y fallos con la arquitectura mínima que permite estudiarlos.

Bibliografía